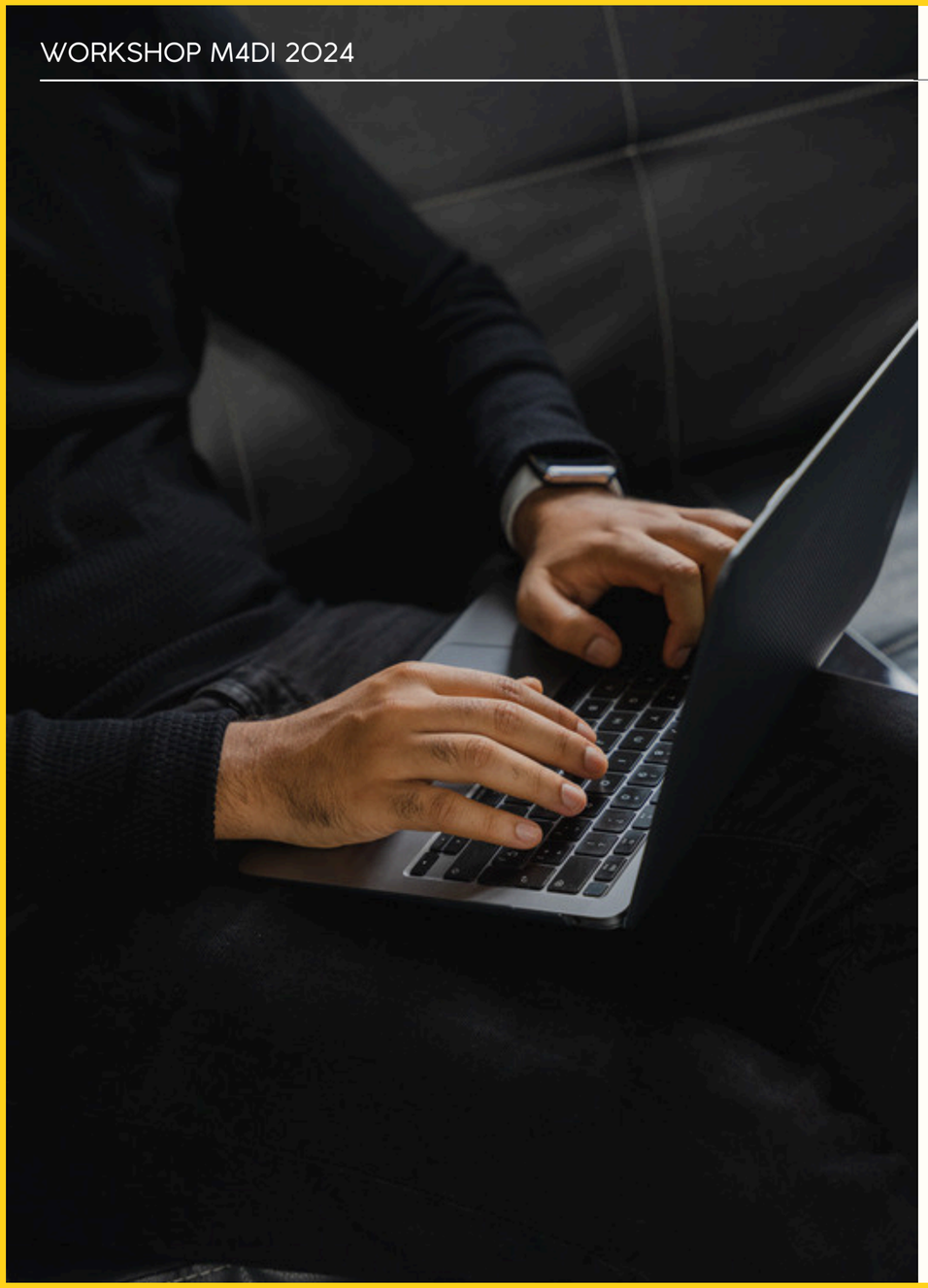
GIT

AND A FEW FAIR PRINCIPLES



PREREQUISITES

- Git installation
- Base commands

1. PIMP MY GIT
2. BEYOND THE BASICS
3. TEAMWORK

1. PIMP MY GIT

A detailed view of the dozens configuration option to ease your Git usage.

git config

```
/
├── bin
├── dev
├── etc/
│   └── gitconfig  <------------------------------  --system
├── usr
├── home/
│   └── user/
│       ├── .config/
│       │   └── git/
│       │       └── config  <------------------  --global
│       └── documents/
│           └── my_super_project/
│               ├── .git/
│               │   └── config  <----------------  --local
│               ├── main.py
│               └── README.py
├── lib
├── sbin
├── tmp
└── var
```

```
init.defaultBranch main
pull.rebase true
```

```
user.name Tux
user.email tux@aol.com
pull.ff-only true
```

```
user.email tux@linux.org
```

```
init.defaultBranch main
pull.ff-only true
user.name Tux
user.email tux@linux.org
```

2. DOTFILES

Those mysterious yet essential files

Main source : https://nesbitt.io/2026/02/05/git-magic-files.html

.gitignore

```
/
├── bin
├── dev
├── etc/
│   └── gitconfig
├── usr
├── home/
│   └── user/
│       ├── .config/
│       │   └── git/
│       │       └── ignore  ←  global
│       └── documents/
│           └── my_super_project/
│               ├── .git/
│               │   └── info
│               │       └── exclude  ←  local-only
│               ├── main.py
│               ├── README.py
│               └── .gitignore  ←  directory-level
├── lib
├── sbin
├── tmp
└── var
```

Valid patterns:

- Wildcards ***.log**
- Directory **dist/**
- Negation **!important.log**
- Globbing

Ignore pattern to apply to everything on the system

Only valid on this machine

Shared with the code

.gitattributes

How git handles files

Interpret file extension

```
*.png binary
```

Consider .png files as binary files: not to be diff or merged, but chose one version.

Define strategies

```
package-lock.json merge=ours
```

Always keep the local version when merging conflicts for package-lock.json files.

Visualise

```
*.json diff=json
```

Use a diff visualiser specific to json files.

.lfsconfig

Big files

LFS : Large File System

Used to save datasets on a forge

.mailmap

Who's that?

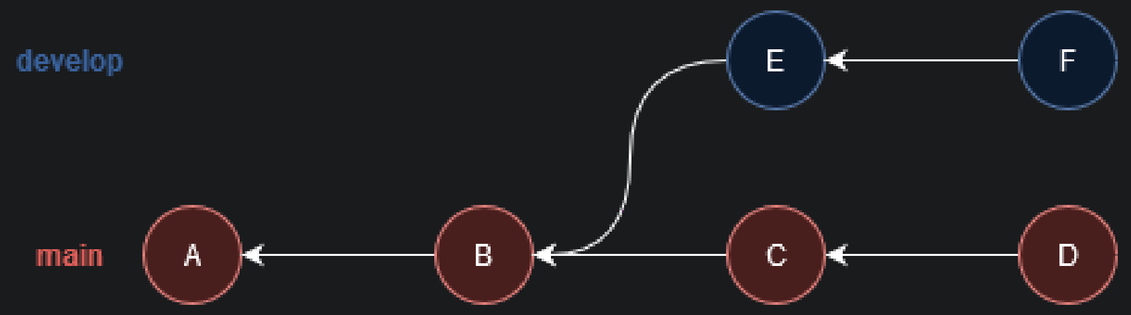
```
Canonical Name <canonical mail> Commit Name
<commit mail>

Benjamin Loire <benjamin.loire@univ-amu.fr>
Rumengol <rumengol29@gmail.com>
```

An exhaustive list here : https://nesbitt.io/2026/02/05/git-magic-files.html

2. BEYOND THE BASICS

Several commands that will change your use of Git

GIT MERGE VS GIT REBASE

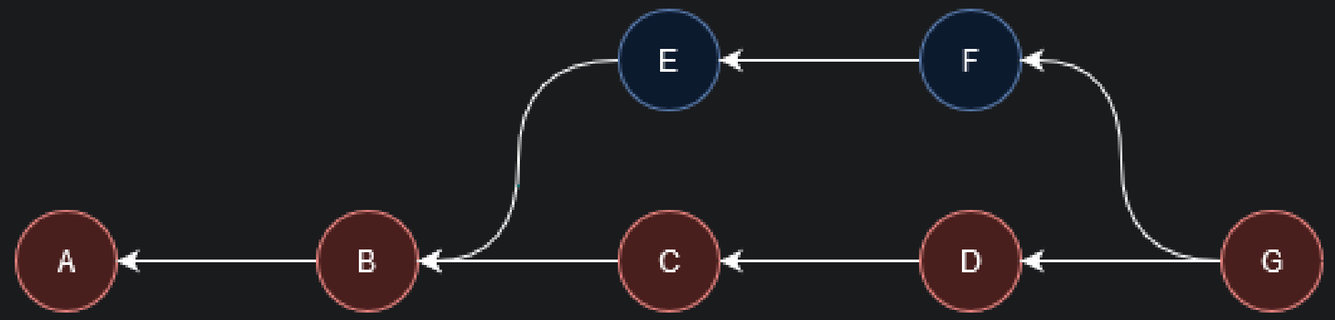


GIT MERGE

```
git merge develop main
```

On branch main

```
git merge develop
```



GIT REBASE

```
git rebase main develop
```

On branch develop

```
git rebase main
```

GIT RESET

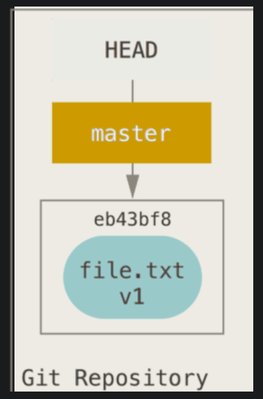
Unstaged changes

GIT ADD

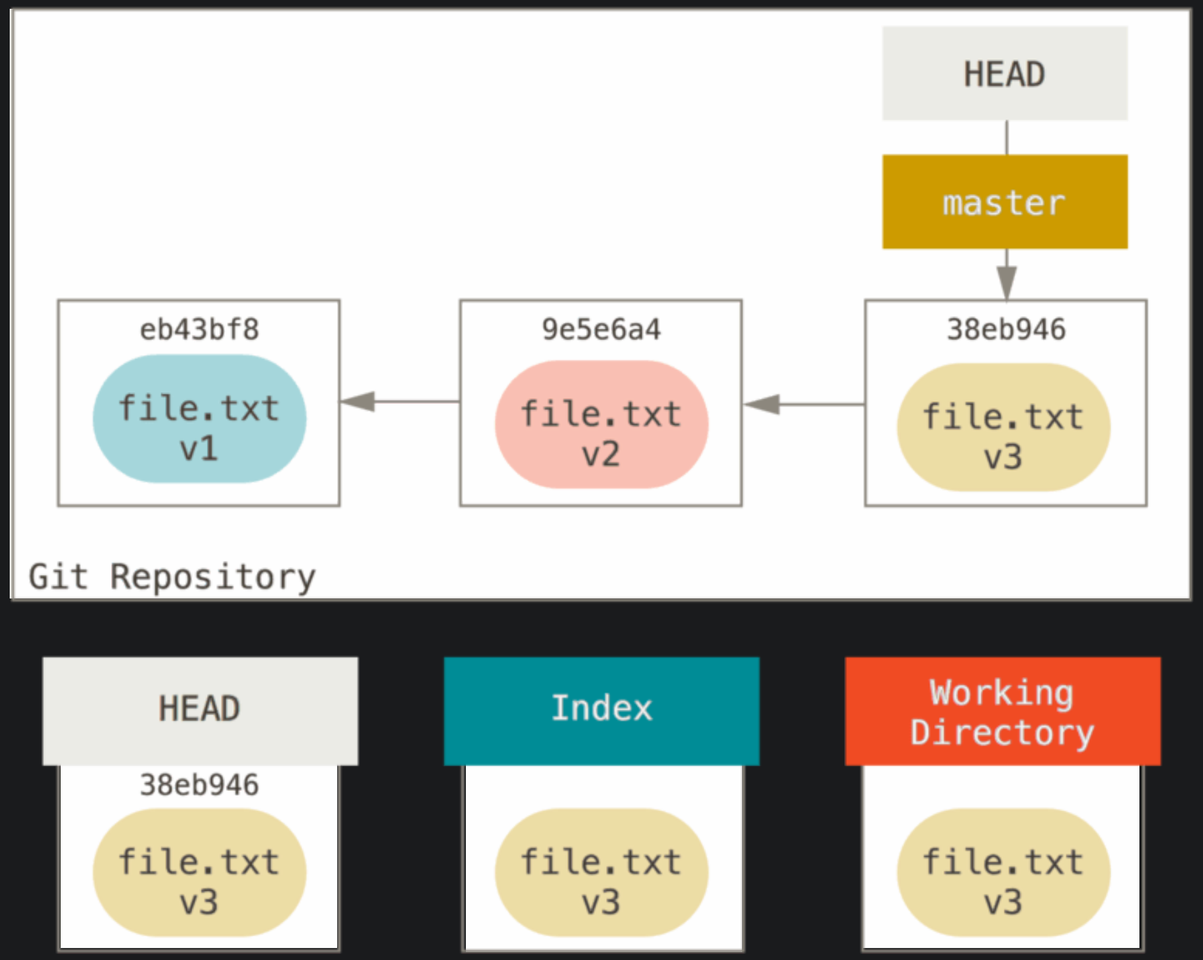
Staged changes

GIT COMMIT

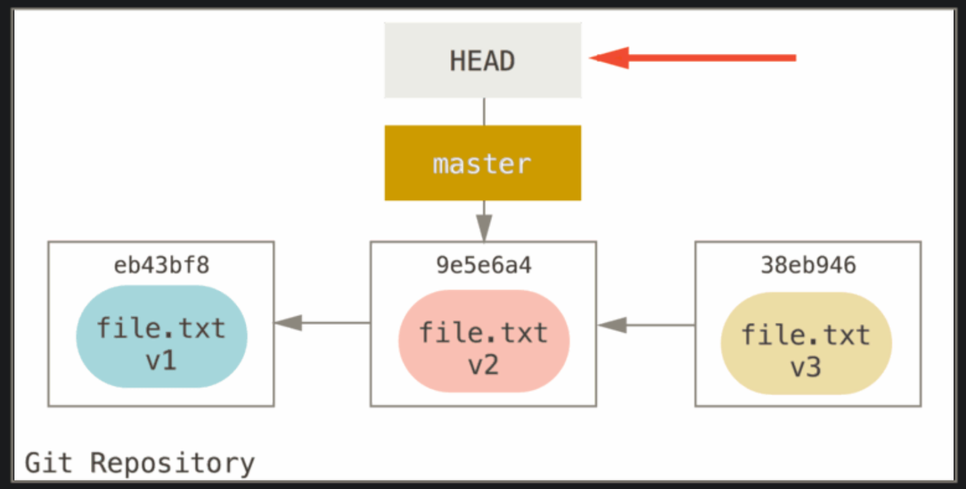
Committed changes



GIT RESET


HEAD
master
eb43bf8
file.txt
v1
9e5e6a4
file.txt
v2
38eb946
file.txt
v3
Git Repository
HEAD
38eb946
file.txt
v3
Index
file.txt
v3
Working
Directory
file.txt
v3

GIT RESET : 1. MOVE HEAD (--SOFT)

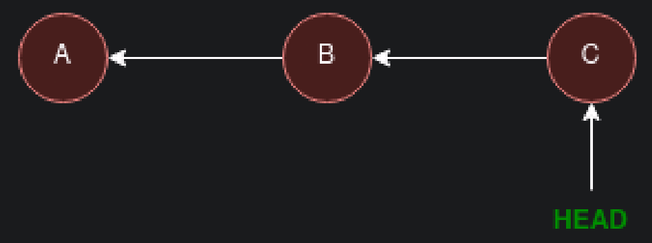


GIT RESET : 2. UPDATE THE INDEX (--MIXED)

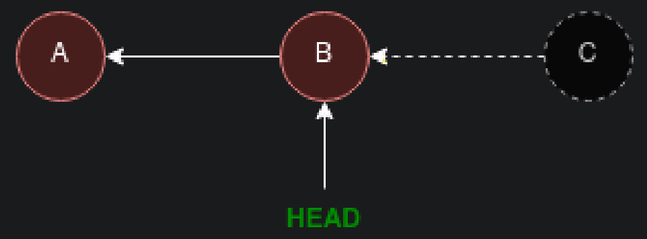
GIT RESET : 3. UPDATE THE WORKSPACE (--HARD)

ANY UNCOMMITTED FILE IS LOST FOREVER!

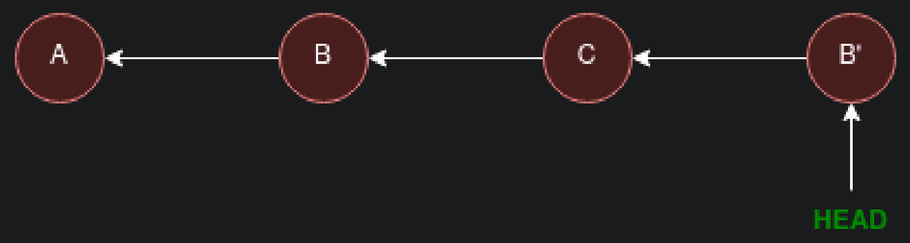
GIT RESET VS GIT REVERT



GIT RESET



GIT REVERT



GIT REFLOG

```
27b5bbd (HEAD -> main, origin/main, origin/HEAD) HEAD@{0}: commit (amend): Filled README
63b3bf7 HEAD@{1}: commit (amend): Filled README
14abc2a HEAD@{2}: reset: moving to 14abc2a
862ae07 HEAD@{3}: reset: moving to 862ae07
14abc2a HEAD@{4}: reset: moving to 14abc2a
6919e76 (feat_revcomp) HEAD@{5}: reset: moving to 6919e76
45181f5 HEAD@{6}: commit: Revert "changed logoden biniou into lorem ipsum"
1ec0892 HEAD@{7}: reset: moving to HEAD
1ec0892 HEAD@{8}: commit (merge): Merge branch 'feature_1'
6919e76 (feat_revcomp) HEAD@{9}: rebase (finish): returning to refs/heads/main
```

EXERCISE !

1. Clone the repo Rumengol/formation_M4DI_2024 (github or gitlab)
2. Rebase the feat_revcomp branch on main
3. Merge the feature_1 branch with main
4. Revert the commit which changes the name of the file to file_v3.txt
5. Reset history two commits back
6. Cancel the reset using git reflog

3. BEST PRACTICES

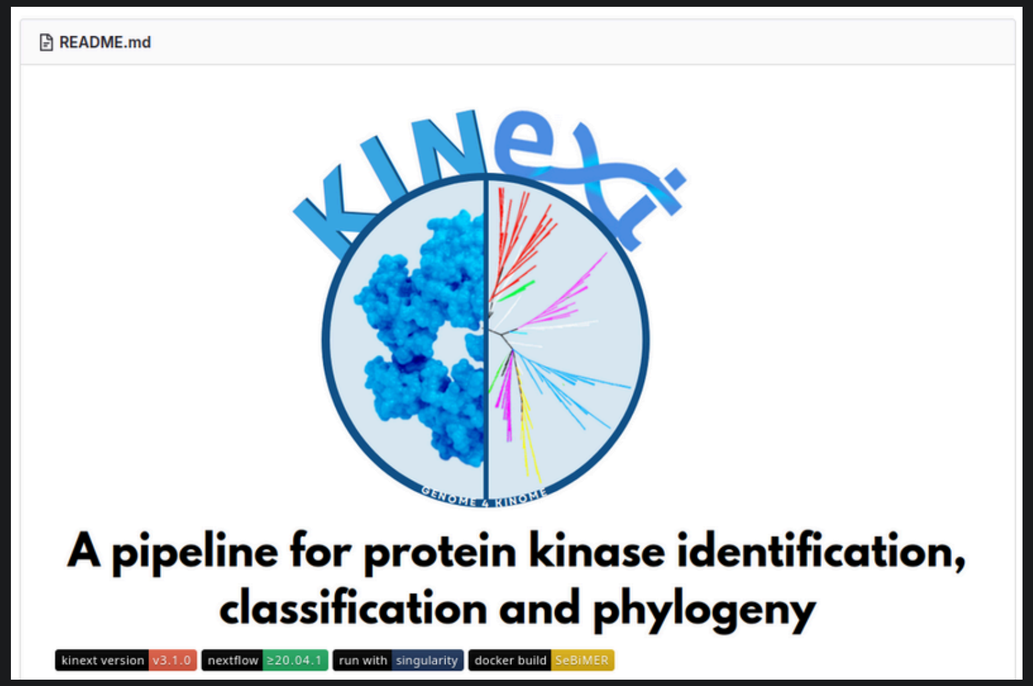
Or how to tame Git to ease its use for you and your team

TEAMWORK: THE FORGE

+ Most famous forge
+ Nearly all bioinfo projects are on there
+ Ideal for small projects with a few team members
- Ethical considerations
- Only 1 cloud instance (or you have to pay)

+ Better CI/CD system
+ Open source, self-hostable instances
+ Ideal for larger projects and bigger teams
- Smaller community
- Complex interface

TEAMWORK: README



KINEXT, ELISABETH HELLEC, IFREMER

https://gitlab.ifremer.fr/bioinfo/workflows/kinnext

- Short one line presentation
- Detailed presentation and background
- Install instructions
- Basic use examples
- License
- Contribution guidelines
- Citation

TEAMWORK: FOLLOW PROGRESS

ISSUES

Open 18 Closed 0 | Author | Labels | Projects | Milestones | Assignees | Types | Newest

- [Feature] Make a preprocessor class and move Node2Vec in it — Complexity: Medium, Enhancement, Priority: High
  Feature #24 · Rumengol opened last month · v1.0.0
- [Dev] Add custom error types — Complexity: Low, Documentation, Miscellaneous, Priority: Low
  Feature #23 · SangaraSorama opened on Jan 23 · V2.0.0
- [Feature] Finish the `run_grid_search` function — Complexity: Medium, Enhancement, Priority: Low
  Feature #21 · SangaraSorama opened on Jan 20
- [Feature] TKGE model: AttBiTi — Blocker, Complexity: Medium, Enhancement, Priority: Medium
  Feature #18 · SangaraSorama opened on Jan 12 · V2.0.0
- [Feature] TKGE model: ChronoR — Blocker, Complexity: Medium, Enhancement, Priority: Medium
  Feature #17 · SangaraSorama opened on Jan 12 · V2.0.0
- [Feature] TKGE model: TNTComplEx — Blocker, Complexity: Medium, Enhancement, Priority: Medium
  Feature #16 · SangaraSorama opened on Jan 12 · V2.0.0
- [Feature] TKGE model: TA-DistMult — Blocker, Complexity: Medium, Enhancement, Priority: Medium
  Feature #15 · SangaraSorama opened on Jan 12 · V2.0.0
- [Feature] TKGE model: TTransE — Blocker, Complexity: Medium, Enhancement, Priority: Medium
  Feature #14 · SangaraSorama opened on Jan 12 · V2.0.0
- [Feature] Temporal Foundations of KGATE — Blocker, Complexity: Medium, Enhancement, Priority: High
  Feature #13 · SangaraSorama opened on Jan 12 · V2.0.0
- [Feature] Temporal Predictors — Blocker, Complexity: Medium, Enhancement, Priority: Medium, Unclear
  Feature #12 · SangaraSorama opened on Jan 12 · V2.0.0
- [Feature] GNNExplainer — Complexity: Medium, Enhancement, Priority: High
  Feature #11 · SangaraSorama opened on Jan 12 · V2.0.0
- [Feature] SHAP Values — Complexity: High, Enhancement, Priority: Medium, Unclear
  Feature #10 · SangaraSorama opened on Jan 12 · V2.0.0

EAMWORK: FOLLOW PROGRESS

PROJECTS (KANBAN VIEW)

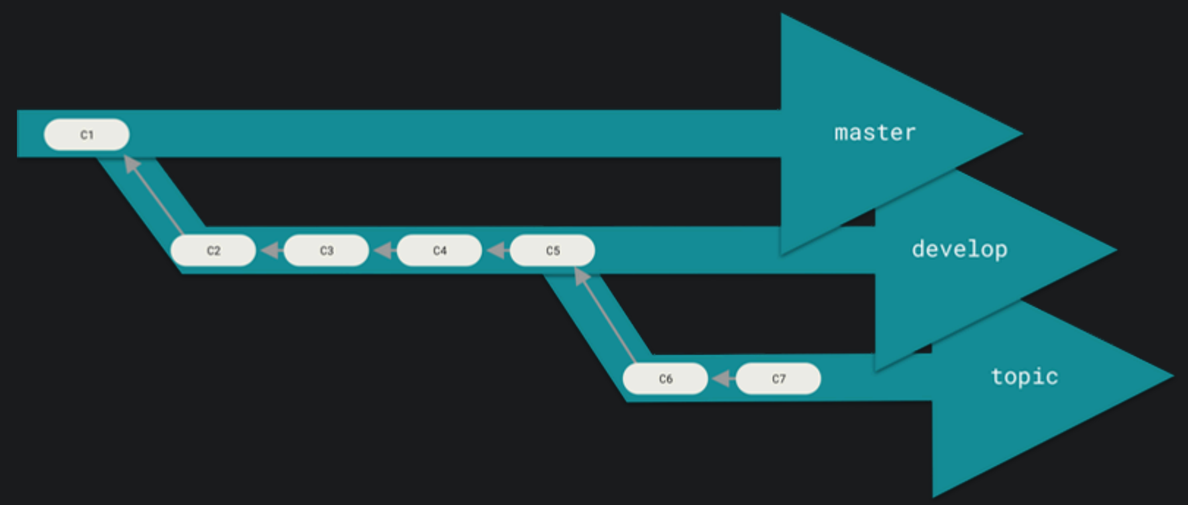
TEAMWORK: COMMITS

RECIPE FOR A GREAT COMMIT MESSAGE

- The first line is the title, or a summary in less than 50 symbols.
- An empty line
- A detailed description with :
  - Commit content
  - Commit purpose
  - Reflections or solutions found in the process

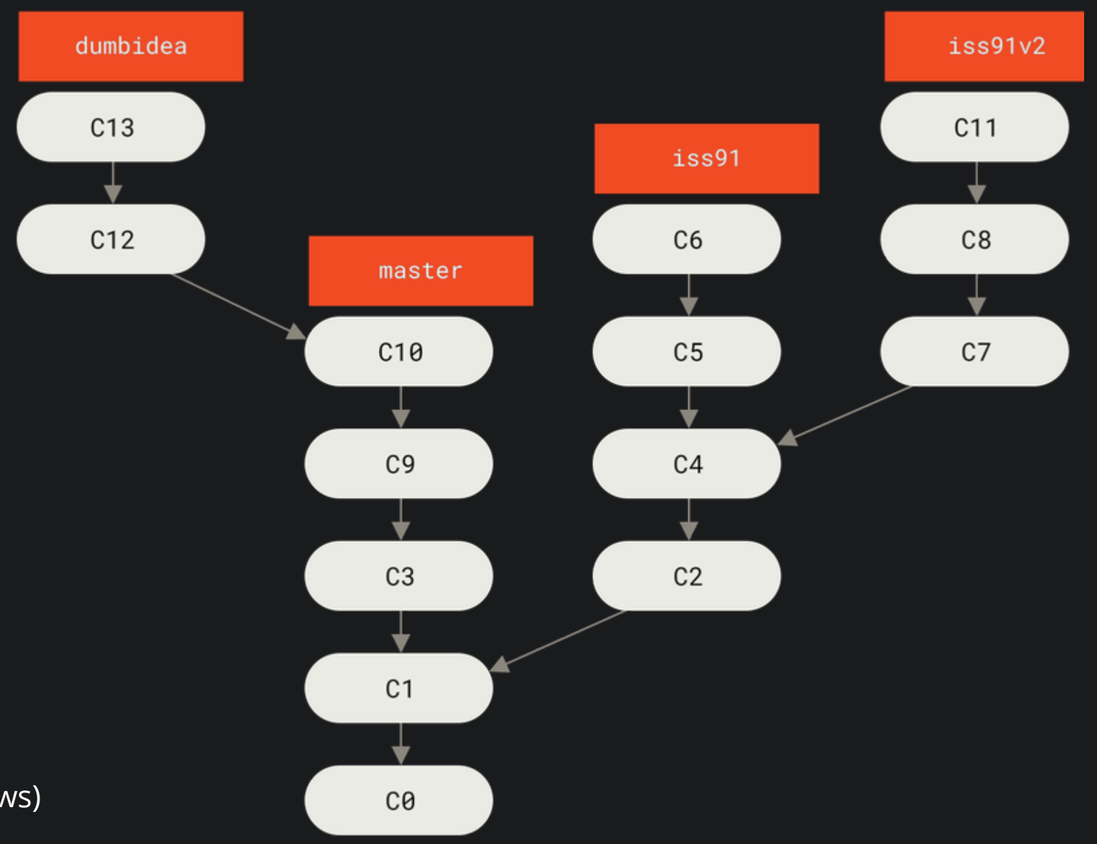
TEAMWORK: BRANCHES

LONG-RUNNING BRANCHES



from the Git book (https://git-scm.com/book/en/v2/Git-Branching-Branching-Workflows)

TOPIC BRANCHES



TEAMWORK: CONFLICTS

Code on the current branch

```
Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
<<< HEAD (Current Change)
        mapping_dataframe = pd.merge(dataframe,
                                     metadata.add_prefix("head_"),
                                     how = "left",
                                     left_on = "head",
                                     right_on = "head_id")
        mapping_dataframe = pd.merge(mapping_dataframe,
                                     metadata.add_prefix("tail_"),
                                     how = "left",
                                     left_on = "tail",
                                     right_on = "tail_id",
                                     suffixes = (None, "_tail"))
        mapping_dataframe.drop([i for i in mapping_dataframe.columns if "id" in i], axis = 1, inplace=True)

        dataframe_node_types = list(set(mapping_dataframe['head_type'].unique()).union(set(mapping_dataframe['tail_type'].unique())))
        self.node_type_to_index = {node_type: i for i, node_type in enumerate(sorted(dataframe_node_types))}
        self._identity = "id" # private attribute, for more info refer to identity property
```

Last common code

```
||| cbb9966    Pin selection to current chat prompt (Ctrl+Alt+X) | Don't show this again (Alt+?)
        assert not set(["type","id"]).isdisjoint(list(metadata.columns)), f"The mapping dataframe must have at least the columns `type

        mapping_df = pd.merge(df, metadata.add_prefix("from_"), how="left", left_on="from", right_on="from_id")
        mapping_df = pd.merge(mapping_df, metadata.add_prefix("to_"), how="left", left_on="to", right_on="to_id", suffixes=(None, "_to
        mapping_df.drop([i for i in mapping_df.columns if "id" in i],axis=1, inplace=True)

        df_node_types = list(set(mapping_df['from_type'].unique()).union(set(mapping_df['to_type'].unique())))
        self.nt2ix = {nt: i for i, nt in enumerate(sorted(df_node_types))}
===
```

Code on the branch to merge

```
        mapping_df = pd.merge(df, metadata.add_prefix("from_"), how="left", left_on="from", right_on="from_id")
        mapping_df = pd.merge(mapping_df, metadata.add_prefix("to_"), how="left", left_on="to", right_on="to_id", suffixes=(None, "_to
        mapping_df.drop([i for i in mapping_df.columns if "id" in i],axis=1, inplace=True)

        df_node_types = list(set(mapping_df['from_type'].unique()).union(set(mapping_df['to_type'].unique())))
        self.nt2ix = {nt: i for i, nt in enumerate(sorted(df_node_types))}
        self._identity = "id"
>>> dev (Incoming Change)
```

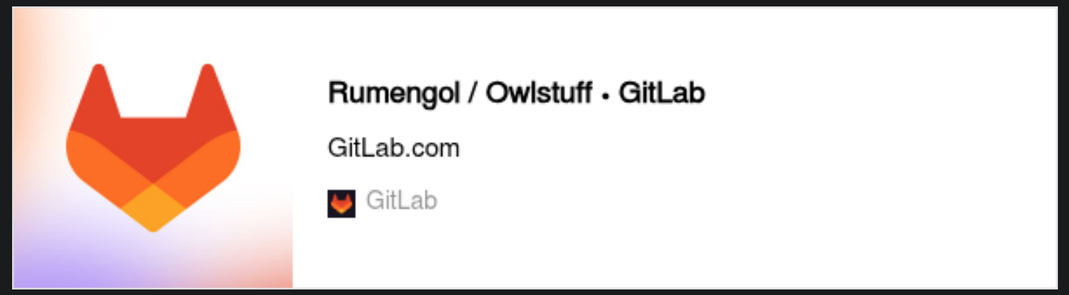
TEAMWORK: CI/CD

CI : CONTINUOUS INTEGRATION

- Run tests for each PR
- Uniformise the code
- Apply conventions (SemVer)

CD : CONTINUOUS DEPLOYMENT

- Deploy an online tool
- Create a new release
- Automate the build



WANT MORE GIT?

https://links.rumengol.net/tag/Git

Benjamin Loire

benjamin.loire@univ-amu.fr